

Yelra Yellow Paper & Formal Specification

Version 0.1 (Draft)

YELRA Project

Last update: November 22, 2025

Abstract

We present the draft specification of the Yelra programming language: design goals, core syntax, static semantics (typing), dynamic semantics (operational), memory model, and conformance requirements. This document is normative where explicitly stated; other sections are informative and provide rationale, examples, and security guidance.

Contents

1	Definition of Terms	3
2	Scope and Non-Goals	5
3	Design Principles	5
4	Implementation Pipeline	5
4.1	<code>lexer.rs</code> – the cutter	5
4.2	<code>parser.rs</code> – the organizer	5
4.3	<code>ast.rs</code> – the boxes/blueprints	6
4.4	<code>eval.rs</code> – the cook	6
5	Parser Walkthrough	6
5.1	Imports	6
5.2	The Parser backpack	7
5.3	New parser (clean up and start at the front)	14
5.4	Peek and Next (look vs. move)	15
5.5	<code>parse_expr</code> (turn cards into a little meaning-tree)	16
5.6	Infix sugar: turn <code>1 + 2</code> into <code>(+ 1 2)</code>	20
5.7	Top-level parse function (one expression only, no leftovers)	21
6	Worked Mini-Example	23
7	Glossary	24

1 Definition of Terms

AST (Abstract Syntax Tree) A tree-structured representation of source code that we build from tokens during parsing. Each node corresponds to a syntactic construct (for example, a number literal, a symbol, or an application/list). The AST abstracts away raw text details like whitespace and exact formatting so later stages (typing, evaluation, optimization, or code generation) can operate on a clean, structural form.

2 Scope and Non-Goals

This yellow paper is intended to be implementable with minimal ambiguity. We target:

- A small, well-specified core calculus (§??) with total, deterministic evaluation.
- A practical surface language (§??) that desugars to the core.
- A sound type system with principal types for the core, and *checked effects* as an extension.

Non-goals for v0.1: concurrency semantics, FFI beyond the **extern** stub, and formal cost semantics.

Remarks 2.1. Use this section to log exclusions so implementers do not over-interpret the v0.1 guarantees.

3 Design Principles

1. **Minimal core, rich edges:** keep the core small (functions, products, sums, recursion), push features to libraries and desugaring.
2. **Deterministic evaluation:** the same program yields the same result on any conforming implementation.
3. **Static clarity first:** errors should be rejected at compile-time where feasible, with precise spans and messages.
4. **Ergonomics by desugaring:** sugar like `pattern match` and pipeline operators lower into the core.

4 Implementation Pipeline

Think of the language like a little bakery that turns a recipe (`1 + 1`) into a cake (2). Each file is a helper with a simple job.

4.1 `lexer.rs` – the cutter

Purpose: chop the raw text into small pieces called *tokens*.

Rationale: computers need neat pieces, not one long string.

Meaning: it reads `"1 + 1"` and cuts it into `["1", "+", "1"]` with labels like `Number`, `Symbol`, `LParen`, `RParen`, etc.

Intuitively: scissors that cut sentences into word-cards.

4.2 `parser.rs` – the organizer

Purpose: take the token cards and build a *structure* (a little tree) that shows how things fit together.

Rationale: `1 + 1` isn't just three cards; it's *add* with two numbers.

Meaning: turns tokens into an *AST* shape like `List([Symbol("+"), Number(1), Number(1)])`.

Intuitively: arrange the word-cards into a mini Lego model that says “this is an add”.

4.3 ast.rs – the boxes/blueprints

Purpose: define the *shapes* of those tree nodes.

Rationale: everyone needs to agree on what a “Number” or “Add list” looks like in code.

Meaning: declares types like `Expr::Number`, `Expr::Symbol`, `Expr::List` so the parser and evaluator talk the same language.

Intuitively: the picture guide that says “this block is a Number, this block is a Symbol”.

4.4 eval.rs – the cook

Purpose: read the AST and *do the thing* (compute a result).

Rationale: after we’ve built the “add” model, someone must actually add the numbers.

Meaning: walks the AST; when it sees `(+ 1 1)`, it returns 2.

Intuitively: the baker who follows the model/recipe and makes the cake.

How they work together

1. We type: `1 + 1`
2. `lexer.rs` cuts it $\rightarrow$ tokens: `Number("1"), Symbol("+"), Number("1")`
3. `parser.rs` organizes it $\rightarrow$ AST: `List([Symbol("+"), Number(1), Number(1)])`
4. `eval.rs` cooks it $\rightarrow$ result: 2
5. `ast.rs` is the shared blueprint they all use to describe the pieces.

That’s the whole pipeline: *cut* $\rightarrow$ *organize* $\rightarrow$ *blueprint-shaped tree* $\rightarrow$ *cook*.

5 Parser Walkthrough

Let’s read this file like a picture book. Yelra is learning to **read math sentences**. This file teaches it how.

5.1 Imports

We grab our Lego pieces

```
1 use crate::ast::Expr;
2 use crate::lexer::Token;
```

“Bring the *Expr* shapes (our Lego models) and the *Token* cards (cut-up words) so we can use them.”

C# equivalent

```
1 // C# namespaces (example)
2 using System;
3 using System.Collections.Generic;
4 using System.Linq;
5
```

```
6 // We'll define Token and Expr types below in C# snippets.
```

5.2 The Parser backpack

What it carries

```
1 pub struct Parser {  
2     tokens: Vec<Token>,  
3     pos: usize,  
4 }
```

The *Parser* holds: (i) *tokens*: the list of word-cards to read; (ii) *pos*: a finger pointing at which card we're on.

Mathematical form:

$\mathbb{T}$ — the set of all possible tokens (*Token*).

Rust code:

```
1 // Universe of tokens (T)  
2 #[derive(Clone, Debug)]  
3 pub enum Token {  
4     LParen,  
5     RParen,  
6     Whitespace,  
7     Number(String),  
8     Symbol(String),  
9 }
```

Plain Language:

“ $\mathbb{T}$ ” is the **universe of all token values** — basically, every kind of symbol our parser can recognize. Think of it as the **alphabet of our language**, just like all letters A–Z form the alphabet for English.

Intuitively:

Imagine a **big box of Lego blocks** — each block shape (circle, square, triangle) is one kind of token. The set $\mathbb{T}$ is the box containing **all block shapes our parser can possibly see**.

Mathematical form:

$\mathbb{L}(\mathbb{T})$ — finite lists (sequences) over $\mathbb{T}$.

Rust code:

```
1 // Lists of tokens (L(T)) as a growable vector
2 ...
3     pub tokens: Vec<Token>
4 ...
```

Plain Language:

“ $\mathbb{L}(\mathbb{T})$ ” means a **list of tokens** — just a sequence of items drawn from $\mathbb{T}$. It’s our program text turned into a stream of small symbols.

Intuitively:

If $\mathbb{T}$ is our box of Lego blocks, then $\mathbb{L}(\mathbb{T})$ is **a row of those blocks clicked together** — like the line of tokens our parser reads one by one.

Mathematical form:

$$\mathbb{N} = \{0, 1, 2, \dots\}$$

Rust code:

```
1 // Natural numbers (N) used for indices and lengths
2 ...
3     pub pos: usize;
4 ...
```

Plain Language:

The **natural numbers** — 0, 1, 2, and so on. We use these to count positions inside the token list.

Intuitively:

These are **the step numbers** on a ruler where our finger can point while reading tokens.

Record (state) type.

Mathematical form:

$$\text{Parser} \stackrel{\text{def}}{=} \{ \text{tokens} : \mathbb{L}(\mathbb{T}), \text{pos} : \mathbb{N} \} \quad \text{with invariant } 0 \leq \text{pos} \leq |\text{tokens}|$$

Rust code:

```
1 // Record/state type with invariant 0 <= pos <= tokens.len()
2 pub struct Parser {
```



```

3   pub tokens: Vec<Token>, // L(T)
4   pub pos: usize,         // N
5 }
6 // (Invariant is enforced by methods; Rust does not encode it in the type)

```

Plain Language:

The **Parser** is defined as a record (like a small box) with:

- a field **tokens**: the list of tokens,
- a field **pos**: the current position (a number).

The **invariant** is a rule that must always be true: the position **pos** must always be **between 0 and the total number of tokens**. Here $|\mathbf{tokens}|$ means “the number of tokens” (the length of the list).

Intuitively:

Imagine a **finger sliding along a row of cards**.

- **tokens** = the cards,
- **pos** = where our finger is,
- $0 \leq \mathbf{pos} \leq |\mathbf{tokens}|$ means our finger is never before the first card or past the last one.

Concrete parser state.

Mathematical form:

$$p = \langle \mathbf{tokens} = t_0, t_1, \dots, t_{n-1}, \mathbf{pos} = k \rangle$$

with $n = |\mathbf{tokens}|$ and $0 \leq k \leq n$.

Rust code:

```

1 pub struct Parser {
2     tokens: Vec<Token>,
3     pos: usize,
4 }

```

Plain Language:

A **specific parser state** **p** means:

- the token list is $[t_0, t_1, \dots, t_{n-1}]$,
- the cursor position is **k** (somewhere between 0 and n).

Intuitively:

Think of a **train of n cars** labeled $t_0 \dots t_{n-1}$. Our finger (**pos**) points at one car — it can start before the first, or end just past the last car, but never leave the track.

Term-by-term explanation (Rust meaning, plain language, intuitively)

Here is a plain-English tour of every symbol, keyword, and bit of jargon in:

```
pub struct Parser { tokens: Vec<Token>, pos: usize, }
```

pub

***Rust meaning:** visibility modifier (public outside the current module).*

***plain language:** anyone in our project can use this.*

***Intuitively:** a classroom project we are allowed to show to the whole school, not just our table.*

struct

***Rust meaning:** defines a composite data type with named fields (a record).*

***plain language:** a custom box with labeled compartments.*

***Intuitively:** a lunchbox with sections labeled “sandwich” and “snack.”*

Parser

***Rust meaning:** the name (identifier) of this struct type.*

***plain language:** the model name of the box we designed.*

***Intuitively:** calling our lunchbox model “CrunchMaster 3000.”*

{ ... } (after **Parser**)

***Rust meaning:** the body of the struct—lists its fields.*

***plain language:** the blueprint that lists the compartments.*

***Intuitively:** a diagram showing which lunchbox sections exist.*

tokens

***Rust meaning:** field name; holds a collection of *Tokens*.*

***plain language:** the pile of word-cards the parser will read.*

***Intuitively:** a stack of flashcards we go through one by one.*

:

***Rust meaning:** type ascription operator; “has type.”*

***plain language:** this thing is of kind...*

Intuitively: a label under a cubby that says “snacks go here.”

Vec<Token>

Rust meaning: growable vector (dynamic array) of elements of type **Token**; angle brackets indicate a generic.

plain language: a resizable list of tokens.

Intuitively: a stretchy accordion folder holding lots of identical flashcards.

Vec

Rust meaning: standard library dynamic array type.

plain language: a resizable array container.

Intuitively: a backpack that magically expands when we add more cards.

<Token> (generic parameter)

Rust meaning: specifies the element type for the vector (generics).

plain language: this is a list of Tokens (not apples, not screws).

Intuitively: a Lego bin labeled “only blue 2-stud bricks.”

Token

Rust meaning: a type for lexical units (numbers, symbols, parentheses, etc.).

plain language: one chopped piece of the original text.

Intuitively: a single flashcard with a symbol or number on it.

, (comma)

Rust meaning: separates fields; trailing commas are allowed.

plain language: next field coming up.

Intuitively: a divider between items in a packing list.

pos

Rust meaning: field name; tracks the current index into **tokens**.

plain language: the cursor/finger pointing at where we are in the list.

Intuitively: our finger marking which flashcard we are reading.

`usize`

***Rust meaning:** unsigned integer sized to the platform's pointer width; ideal for indexing.*

***plain language:** a whole number sized for our machine, used for positions/lengths.*

***Intuitively:** a page number big enough for any book our shelf can hold.*

`}` (closing brace), no semicolon

***Rust meaning:** ends the struct body; no semicolon after a type definition.*

***plain language:** blueprint complete.*

***Intuitively:** closing the lid on the lunchbox plan.*

Invariant summary

Mathematical form:

- **Bounds:** $0 \leq pos \leq |tokens|$
- **End-of-input:** $pos = |tokens|$ iff `peek` and `next` are undefined.

Plain Language:

- The position is always within bounds.
- When the cursor reaches the end ($pos = |tokens|$), both `peek` and `next` can't do anything anymore.

Intuitively:

Our finger is always between start and end of the track. When our finger is after the last card, we can't peek or take the next one — the deck is empty.

Type-theory style (for completeness)

Mathematical form:

$$\text{Parser} \equiv \Sigma(\text{tokens} : \mathbb{L}(\mathbb{T})). \Sigma(pos : \mathbb{N}). (0 \leq pos \leq |tokens|)$$

Plain Language:

*This is the **type-theory definition** — it says a `Parser` is a dependent pair (Sigma-type) that contains:*

1. a list of tokens,
2. a position number,
3. plus a **proof** that the position satisfies $0 \leq pos \leq |tokens|$.

The Σ symbol means “there exists a pair of values with this property.”

Intuitively:

Imagine a **sealed box** that contains:

- the row of tokens,
- our finger’s index,
- and a **note certifying** that “the finger is safely within range.” Every valid parser must carry that note.

Mini big-picture

- The **struct** defines a **Parser** that owns two things:
 - **tokens**: a resizable list of **Token** pieces to read.
 - **pos**: a position counter (a machine-sized, non-negative integer) showing where we are looking.
- **Public** (**pub**) means other modules/crates can create and use this parser type.
- **Generics** (**Vec<Token>**) means “a container that specifically holds **Tokens**.”

If we keep the lunchbox image in mind: we built a public model called **Parser** with two compartments—one stretchy compartment for all the flashcards (**tokens**) and one little window showing the current card number (**pos**).

C# equivalent

```
1 // Minimal algebraic-style modeling for tokens and expressions
2 public abstract record Token;
3 public sealed record LParenToken() : Token;
4 public sealed record RParenToken() : Token;
5 public sealed record WhitespaceToken() : Token;
6 public sealed record NumberToken(string Text) : Token;
7 public sealed record SymbolToken(string Text) : Token;
8
9 public abstract record Expr;
10 public sealed record Number(double Value) : Expr;
11 public sealed record Symbol(string Name) : Expr;
12 public sealed record ListExpr(List<Expr> Items) : Expr;
13
14 public sealed class Parser
15 {
16     private readonly List<Token> tokens;
17     private int pos;
18
19     public Parser(IEnumerable<Token> tokens)
20     {
21         // whitespace is removed here (same as Rust new())
22         this.tokens = tokens.Where(t => t is not WhitespaceToken).ToList();
```

```

23         this.pos = 0;
24     }
25     // methods follow...
26 }

```

5.3 New parser (clean up and start at the front)

```

1  impl Parser {
2      pub fn new(tokens: Vec<Token>) -> Self {
3          let tokens: Vec<Token> = tokens
4              .into_iter()
5              .filter(|t| !matches!(t, Token::Whitespace))
6              .collect();
7
8          Parser { tokens, pos: 0 }
9      }

```

When we make a new parser: we throw away whitespace cards (spaces/tabs/newlines) so we don't trip over them, and set our finger (*pos*) to the start.

C# equivalent

```

1  // Already shown in the C# constructor above; same behavior:
2  // - filters out WhitespaceToken
3  // - sets pos = 0

```

Accessors

Mathematical form:

$$\text{tokens}(p) \stackrel{\text{def}}{=} p.\text{tokens}, \quad \text{pos}(p) \stackrel{\text{def}}{=} p.\text{pos}.$$

Plain Language:

These are *getter functions*:

- *tokens(p)* means “the token list inside parser *p*.”
- *pos(p)* means “the current cursor position inside parser *p*.”

Intuitively:

If *p* is a *toy box*, then *tokens(p)* is *the row of toys inside*, and *pos(p)* is *the sticker showing where our finger is pointing*.

5.4 Peek and Next (look vs. move)

Useful partial operations

peek

Mathematical form:

$$\text{peek} : \text{Parser} \rightarrow \mathbb{T}, \quad \text{peek}(p) = \begin{cases} \text{tokens}(p)[, \text{pos}(p),] & \text{if } \text{pos}(p) < |\text{tokens}(p)|, \\ \text{undefined} & \text{otherwise.} \end{cases}$$

Plain Language:

- The arrow ($\rightarrow$) means “**partial function**” — peek doesn’t always work.
- $\text{peek}(p)$ looks at the current token at position $\text{pos}(p)$ **without moving** the cursor.
- If we’re already at the end ($\text{pos}(p) = |\text{tokens}(p)|$), there’s nothing left to see — so it’s **undefined**.

Intuitively:

“Peek” = **look at the next candy** in the box without taking it out. If the box is empty, we can’t peek — there’s no candy to look at.

```

1  fn peek(&self) -> Option<Token> {
2      self.tokens.get(self.pos)
3  }
```

peek = “Look at the current card, but don’t move our finger.”

next

Mathematical form:

$$\text{next} : \text{Parser} \rightarrow (\mathbb{T} \times \text{Parser}), \quad \text{next}(p) = \begin{cases} (\text{tokens}(p)[, k,], p[\text{pos} \mapsto k+1]) & \text{if } k = \text{pos}(p) < |\text{tokens}(p)|, \\ \text{undefined} & \text{otherwise.} \end{cases}$$

where $p[\text{pos} \mapsto k+1]$ denotes the record p with the pos field updated to $k+1$.

Plain Language:

- $\text{next}(p)$ returns **a pair** (written as $\mathbb{T} \times \text{Parser}$):
 1. the current token,
 2. the **new parser** after moving one step forward.
- The notation $p[\text{pos} > k+1]$ means: “take parser p and change its position to $k+1$.”
- It’s undefined if we’re already at the end (no next token).

Intuitively:

“Next” = **take the next candy and move our finger forward** by one spot. If there’s no candy left, we can’t take any — the operation fails.

```

1  fn next(&mut self) -> Option<Token> {
2      if self.pos < self.tokens.len() {
3          let t = self.tokens[self.pos].clone();
4          self.pos += 1;
5          Some(t)
6      } else {
7          None
8      }
9  }

```

next = “Give us the current card and move our finger to the next card.” If we’re past the last card, we get *None*.

C# equivalent

```

1  private Token? Peek() => pos < tokens.Count ? tokens[pos] : null;
2
3  private Token? Next()
4  {
5      if (pos < tokens.Count) return tokens[pos++];
6      return null;
7  }

```

5.5 parse_expr (turn cards into a little meaning-tree)

```

1  pub fn parse_expr(&mut self) -> Result<Expr, String> {
2      // Parse a primary expression first
3      let first = match self.next() {

```

We’re going to read one expression. First, read a primary piece (a number, a symbol, or a parenthesized list).

5.5.0.1 Case: Number

```

1      Some(Token::Number(s)) => {
2          let n: f64 = s
3              .parse()
4              .map_err(|e| format!("Invalid number '{}': {}", s, e))?;
5          Expr::Number(n)
6      }

```

If the card says *Number("123")*, turn it into *Expr::Number(123.0)*; otherwise complain.

5.5.0.2 Case: bare Symbol


```

1      Some(Token::Symbol(s)) => {
2          // A bare symbol (not inside parentheses) -- return as symbol
3          Expr::Symbol(s)
4      }

```

If the card says `Symbol("+")` or `Symbol("x")`, make a `Expr::Symbol("...")`. We don't know usage yet.

5.5.0.3 Case: List starting with '('

```

1      Some(Token::LParen) => {
2          // parse list until matching RParen
3          let mut exprs = Vec::new();
4          while let Some(tok) = self.peek() {
5              if matches!(tok, Token::RParen) {
6                  // consume RParen
7                  self.next();
8                  return Ok(Expr::List(exprs));
9              } else {
10                 let e = self.parse_expr()?;
11                 exprs.push(e);
12             }
13         }
14         return Err("Unclosed '(' -- reached end of input".to_string());
15     }

```

Seeing (starts a list: read expressions until). Inside, we recursively call `parse_expr()` for each item. If we never find), error.

5.5.0.4 Case: Unexpected ')', whitespace, or end

```

1      Some(Token::RParen) => return Err("Unexpected ')'.to_string()),
2      Some(Token::Whitespace) => return Err("Unexpected whitespace token".to_string
3      ()),
4      None => return Err("Unexpected end of input".to_string()),
5      };

```

A lone) is bad; whitespace shouldn't appear (we filtered it); and empty input is an error.

C# equivalent

```

1  public bool TryParseExpr(out Expr expr, out string? error)
2  {
3      // helper to emit errors
4      Expr? PrimaryFromToken(Token? tok, out string? err)
5      {
6          err = null;

```

```

7      if (tok is null) { err = "Unexpected end of input"; return null; }
8
9      switch (tok)
10     {
11         case NumberToken nt:
12             if (!double.TryParse(nt.Text, out var val))
13             {
14                 err = $"Invalid number '{nt.Text}'";
15                 return null;
16             }
17             return new Number(val);
18
19         case SymbolToken st:
20             return new Symbol(st.Text);
21
22         case LParenToken:
23             {
24                 var items = new List<Expr>();
25                 while (true)
26                 {
27                     var look = Peek();
28                     if (look is RParenToken)
29                     {
30                         Next(); // consume ')'
31                         return new ListExpr(items);
32                     }
33                     var ok = TryParseExpr(out var inner, out var innerErr);
34                     if (!ok) { err = innerErr; return null; }
35                     items.Add(inner);
36
37                     if (Peek() is null)
38                     {
39                         err = "Unclosed '(' -- reached end of input";
40                         return null;
41                     }
42                 }
43             }
44
45         case RParenToken:
46             err = "Unexpected ')'";
47             return null;
48
49         case WhitespaceToken:
50             err = "Unexpected whitespace token";

```

```

51         return null;
52
53     default:
54         err = "Unknown token";
55         return null;
56     }
57 }
58
59 // 1) parse primary
60 var firstToken = Next();
61 var first = PrimaryFromToken(firstToken, out error);
62 if (first is null) { expr = null!; return false; }
63
64 // 2) infix sugar: only if first is Number or ListExpr
65 if (first is Number || first is ListExpr)
66 {
67     var operands = new List<Expr> { first };
68     var ops = new List<string>();
69
70     // collect (op, rhs)* while next token is a Symbol
71     while (Peek() is SymbolToken)
72     {
73         var opTok = Next() as SymbolToken; // consume operator
74         var ok = TryParseExpr(out var rhs, out var rhsErr);
75         if (!ok) { expr = null!; error = rhsErr; return false; }
76
77         ops.Add(opTok!.Text);
78         operands.Add(rhs);
79     }
80
81     if (ops.Count == 0)
82     {
83         expr = operands[0]; error = null; return true;
84     }
85     else if (ops.Count == 1)
86     {
87         var list = new List<Expr>();
88         list.Add(new Symbol(ops[0]));
89         list.AddRange(operands);
90         expr = new ListExpr(list); error = null; return true;
91     }
92     else
93     {
94         var same = ops.All(o => o == ops[0]);

```

```

95         if (!same)
96         {
97             expr = null!; error = "Mixed operators without parentheses are not
supported -- use parentheses to disambiguate.";
98             return false;
99         }
100         var list = new List<Expr>();
101         list.Add(new Symbol(ops[0]));
102         list.AddRange(operands);
103         expr = new ListExpr(list); error = null; return true;
104     }
105 }
106
107 // first was a Symbol: return as-is (no infix folding)
108 expr = first; error = null; return true;
109 }

```

5.6 Infix sugar: turn 1 + 2 into (+ 1 2)

```

1      // If the primary expression is a Number or a List, try to parse infix
continuation:
2      // pattern: first (Symbol op, Expr rhs)+
3      match &first {
4          Expr::Number(_) | Expr::List(_) => {
5              let mut operands: Vec<Expr> = vec![first.clone()];
6              let mut ops: Vec<String> = Vec::new();
7
8              // collect (op, rhs) pairs greedily, but only when the next token is a
Symbol
9              while let Some(Token::Symbol(_)) = self.peek() {
10                 // consume operator symbol
11                 let op = match self.next() {
12                     Some(Token::Symbol(s)) => s,
13                     _ => unreachable!(),
14                 };
15                 // parse rhs expression
16                 let rhs = self.parse_expr()?;
17                 ops.push(op);
18                 operands.push(rhs);
19             }
20
21             if ops.is_empty() {
22                 // no infix continuation; just return the primary expr
23                 Ok(operands.into_iter().next().unwrap())

```

```

1      } else if ops.len() == 1 {
2          // single operator: (op first rhs)
3          let op = ops[0].clone();
4          let mut list: Vec<Expr> = Vec::new();
5          list.push(Expr::Symbol(op));
6          list.extend(operands);
7          Ok(Expr::List(list))

```

```

1      } else {
2          // multiple operators: ensure they are all the same (left-assoc, same
- op only)
3          let all_same = ops.iter().all(|o| o == &ops[0]);
4          if all_same {
5              let op0 = ops[0].clone();
6              let mut list: Vec<Expr> = Vec::new();
7              list.push(Expr::Symbol(op0));
8              list.extend(operands);
9              Ok(Expr::List(list))
10         } else {
11             Err("Mixed operators without parentheses are not supported -- use
parentheses to disambiguate.".to_string())
12         }
13     }

```

```

1      }
2      // If the first expression is a Symbol (e.g. user typed "+ 1 2" without
parentheses),
3      // don't attempt to treat following tokens as infix; just return the symbol
expression.
4      Expr::Symbol(_) => Ok(first),
5      }
6  }

```

If we saw many operators (like $1 + 2 + 3$), we only allow it if they're all the same (e.g., $+$), producing $(+ 1 2 3)$. Mixed operators (e.g., $+$ and $*$) are rejected—use parentheses.

C# equivalent

```

1 // Covered inside TryParseExpr above.
2 // The same "all operators must match" rule is enforced with LINQ .All(...).

```

5.7 Top-level parse function (one expression only, no leftovers)

```

1 // top-level parse entry
2 pub fn parse(tokens: Vec<Token>) -> Result<Expr, String> {
3     let mut p = Parser::new(tokens);
4     let expr = p.parse_expr()?;

1     // Ensure we've consumed all tokens; if anything remains, report where we stopped.
2     if let Some(remaining) = p.peek() {
3         // Give a clearer message for debugging leftover tokens
4         let kind = match remaining {
5             Token::LParen => "'(')".to_string(),
6             Token::RParen => "')'".to_string(),
7             Token::Number(n) => format!("number '{}'", n),
8             Token::Symbol(s) => format!("symbol '{}'", s),
9             Token::Whitespace => "whitespace".to_string(),
10        };
11        return Err(format!(
12            "Extra tokens after first expression (next token: {})",
13            kind
14        ));
15    }

```

```

1     Ok(expr)
2 }

```

We parse one expression into a tree and then ensure nothing remains; if there are leftovers, we report what's next for easier debugging.

C# equivalent

```

1 public static Expr ParseOne(IEnumerable<Token> tokens)
2 {
3     var p = new Parser(tokens);
4     if (!p.TryParseExpr(out var expr, out var err))
5         throw new InvalidOperationException(err);
6
7     var remaining = p.Peek();
8     if (remaining is not null)
9     {
10        var kind = remaining switch
11        {
12            LParenToken => "'(')",
13            RParenToken => "')'",
14            NumberToken n => $"number '{n.Text}'",
15            SymbolToken s => $"symbol '{s.Text}'",
16            WhitespaceToken => "whitespace",

```

```

17         - => "unknown"
18     };
19     throw new InvalidOperationException(
20         $"Extra tokens after first expression (next token: {kind})");
21 }
22 return expr;
23 }

```

Summary of main notations:

<i>Symbol</i>	<i>Meaning</i>	<i>Plain Language</i>	<i>intuitively</i>
$\mathbb{T}$	Set of tokens	Token type	Box of Lego pieces
$\mathbb{L}(\mathbb{T})$	Lists of tokens	Token list	Row of Lego blocks
$\mathbb{N}$	Natural numbers	Index / position	Counting steps
(tokens)			Length of token list <code>.len()</code> Number of blocks
$\stackrel{\text{def}}{=}$	“Is defined as”	Definition	Labeling a box
$\rightarrow$	Partial function	May fail	Works only sometimes
$\times$	Pair (tuple)	Returns two things	Candy + new box
$\mapsto$	Field update	Assign new value	Move finger to next card
Σ	Dependent pair	Record with proof	Safe box with rule attached
<i>iff</i>	“If and only if”	Exact equivalence	Two-way condition

Summary

- We throw away spaces first.
- We read a number, a symbol, or a $(\dots)$ list.
- We allow $1 + 2 (+ 3 \dots)$ by turning it into the list $(+ 1 2 3)$ when all ops match.
- Mixed ops like $+$ and $*$ together are rejected—use parentheses.
- We parse exactly one expression and make sure nothing is left behind.

6 Worked Mini-Example

```

1 // factorial via fix
2 fn fact(n: Int): Int {
3   (fix f. fn k: Int {
4     if k == 0 then 1 else k * f(k - 1)
5   })(n)
6 }

```

Typing sketch: $\vdash \text{fact} : \text{Int} \rightarrow \text{Int}$. Evaluation follows E-APP1 and E-LET-style reductions with E-APP1 substituting into the fixed point.

7 Glossary

Core *The minimal typed lambda calculus Yelra is based on.*

Desugaring *Translation from surface syntax to core terms.*

Progress/Preservation *The two pillars of type soundness.*

Acknowledgments

We thank contributors who refined the lexer/parser and clarified edge cases in desugaring.